

Data Visualisation

Python — Matplotlib, Seaborn & Plotly

1 Why Visualise? — The purpose of charts in data science

2 Libraries overview — Matplotlib, Seaborn, Plotly

3 Matplotlib basics — Figure, Axes & pyplot

4 Line plot — plt.plot()

5 Bar chart — plt.bar() / plt.barh()

6 Histogram — plt.hist()

7 Scatter plot — plt.scatter()

8 Pie chart — plt.pie()

9 Customisation — titles, labels, legends, colours, styles

1
0 Subplots — plt.subplots()

1
1 Seaborn — statistical plots

1
2 Plotly — interactive charts

1
3 Choosing the right chart

1
4 Quick reference — chart types & when to use them

1 · Why Visualise?

- Humans process visual patterns far faster than raw numbers.
- **EDA (Exploratory Data Analysis)** — spot distributions, outliers, and correlations quickly.
- Communicate insights clearly to non-technical stakeholders.
- Validate model outputs and data quality (e.g. check for skew before modelling).
- A single chart can reveal what hours of looking at a table cannot.

2 · Libraries Overview

Library	Best For	Style	Interactivity
Matplotlib	Full control, publication plots	Low-level, verbose	Static
Seaborn	Statistical visualisation	High-level over Matplotlib	Static

Plotly	Interactive dashboards & web	High-level	Interactive (zoom, hover)
--------	------------------------------	------------	---------------------------

```
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

# Seaborn is built on top of Matplotlib
# Plotly runs in browsers / Jupyter notebooks
```

3 · Matplotlib Basics — Figure, Axes & pyplot

Every Matplotlib plot lives inside a **Figure** (the whole canvas) containing one or more **Axes** (individual plot areas).

```
import matplotlib.pyplot as plt

# Quick functional approach (pyplot)
plt.plot([1, 2, 3], [4, 5, 6])
plt.show()

# Object-oriented approach (recommended for complex plots)
fig, ax = plt.subplots() # fig = canvas, ax = plot area
ax.plot([1, 2, 3], [4, 5, 6])
plt.show()
```

■ Prefer the object-oriented (fig, ax) approach when building multi-panel figures or when customising heavily. The pyplot (plt.) approach is fine for quick single plots.

4 · Line Plot — plt.plot()

Used to show **trends over time** or continuous relationships between two variables.

```
import matplotlib.pyplot as plt

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May']
revenue = [200, 250, 180, 320, 290]

plt.plot(months, revenue, color='steelblue', linewidth=2,
         marker='o', linestyle='--', label='Revenue')
plt.title('Monthly Revenue')
plt.xlabel('Month')
plt.ylabel('Revenue ($)')
plt.legend()
plt.grid(True)
plt.show()
```

Parameter	Purpose	Example values
color	Line colour	'red', '#2ecc71', 'steelblue'
linewidth	Thickness of line	1, 2, 3
linestyle	Dash pattern	'-', '--', ':-', ':'
marker	Point shape	'o', 's', '^', 'x', 'D'

label	Legend entry text	'Revenue', 'Sales'
-------	-------------------	--------------------

5 · Bar Chart — plt.bar() / plt.barh()

Used to **compare quantities** across discrete categories. Use `barh()` for horizontal bars when category names are long.

```
categories = ['Python', 'R', 'SQL', 'Java']
counts     = [45, 20, 30, 10]

# Vertical bar chart
plt.bar(categories, counts, color='coral', edgecolor='black', width=0.6)
plt.title('Language Popularity')
plt.ylabel('Number of Users')
plt.show()

# Horizontal bar chart
plt.barh(categories, counts, color='steelblue')
plt.xlabel('Number of Users')
plt.show()

# Grouped bar chart (using NumPy for offsets)
import numpy as np
x = np.arange(len(categories))
plt.bar(x - 0.2, counts, width=0.4, label='2023')
plt.bar(x + 0.2, [50,25,35,12], width=0.4, label='2024')
plt.xticks(x, categories)
plt.legend()
```

6 · Histogram — plt.hist()

Shows the **distribution / frequency** of a continuous variable by grouping values into bins.

```
import numpy as np

data = np.random.randn(1000) # 1000 values from normal distribution

plt.hist(data, bins=30, color='mediumseagreen', edgecolor='white',
         density=False)      # density=True shows probability instead of count
plt.title('Distribution of Values')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```

Parameter	Purpose
bins	Number of bars (more bins = finer detail; fewer = smoother)
density	If True, y-axis shows probability density instead of count
cumulative	If True, shows cumulative distribution
color	Fill colour of the bars
edgecolor	Border colour between bars (use 'white' to separate them)

■ A histogram is NOT a bar chart — there are no gaps between bars (unless `edgecolor` is set). Use it only for continuous data. For categorical counts, use a bar chart.

7 · Scatter Plot — `plt.scatter()`

Shows the **relationship / correlation** between two continuous variables. Each point is one observation.

```
import numpy as np

x = np.random.rand(100)
y = 2 * x + np.random.randn(100) * 0.2  # positive correlation

plt.scatter(x, y,
            c='steelblue',      # colour
            s=50,               # marker size
            alpha=0.7,          # transparency
            edgecolors='k')
plt.title('Scatter Plot - X vs Y')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()

# Colour-code by a third variable
plt.scatter(x, y, c=y, cmap='viridis', s=50)
plt.colorbar(label='Y value')
```

■ Use `alpha` (transparency) when points overlap heavily. Add a trendline with `np.polyfit()` or use `seaborn.regplot()` which draws the regression line automatically.

8 · Pie Chart — `plt.pie()`

Shows **part-to-whole relationships** as proportions of a circle. Best for 2–5 categories only.

```
labels = ['Python', 'R', 'SQL', 'Other']
sizes = [45, 20, 25, 10]
explode = (0.05, 0, 0, 0)  # slightly pull out first slice

plt.pie(sizes,
        labels=labels,
        explode=explode,
        autopct='%1.1f%%',  # show percentage inside each slice
        startangle=140,     # rotate start
        colors=['#3498db', '#e74c3c', '#2ecc71', '#f39c12'])
plt.title('Language Market Share')
plt.show()
```

■ Avoid pie charts with more than 5 slices — they become hard to read. A bar chart is almost always a clearer alternative. Use a donut chart (set `wedgeprops={'width':0.5}`) for a modern look.

9 · Customisation — Titles, Labels, Legends, Colours & Styles

```

# Titles and axis labels
plt.title('My Chart', fontsize=16, fontweight='bold', pad=12)
plt.xlabel('X Axis Label', fontsize=12)
plt.ylabel('Y Axis Label', fontsize=12)

# Axis limits and ticks
plt.xlim(0, 100)
plt.ylim(0, 500)
plt.xticks(rotation=45)          # rotate x-axis labels

# Legend
plt.legend(loc='upper right', fontsize=10, framealpha=0.5)

# Grid
plt.grid(True, linestyle='--', alpha=0.5)

# Built-in styles
plt.style.use('seaborn-v0_8-whitegrid') # clean white background
plt.style.use('ggplot')                 # R-style theme
plt.style.use('dark_background')        # dark theme

# Figure size and resolution
plt.figure(figsize=(10, 6), dpi=150)

# Save to file
plt.savefig('chart.png', dpi=300, bbox_inches='tight')

```

10 · Subplots — plt.subplots()

Display **multiple charts side-by-side** in a single figure using a grid of axes.

```

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 5))

# Plot on first axes
axes[0].plot([1,2,3], [4,5,6])
axes[0].set_title('Line Plot')

# Plot on second axes
axes[1].bar(['A', 'B', 'C'], [10,20,15])
axes[1].set_title('Bar Chart')

plt.tight_layout() # prevent overlapping labels
plt.show()

# 2x2 grid
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes[0, 0].plot(...) # top-left
axes[0, 1].bar(...)  # top-right
axes[1, 0].hist(...)  # bottom-left
axes[1, 1].scatter(...) # bottom-right

```

■ Always call `plt.tight_layout()` before `plt.show()` to automatically adjust spacing and prevent titles or labels from overlapping.

11 · Seaborn — Statistical Plots

Seaborn wraps Matplotlib and adds **statistical plotting** with beautiful defaults. It works natively with Pandas DataFrames.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Dataset (built-in Seaborn datasets for practice)
tips = sns.load_dataset('tips')

# Distribution plot – histogram + KDE curve
sns.histplot(tips['total_bill'], kde=True, color='steelblue')

# Box plot – shows median, quartiles, outliers
sns.boxplot(x='day', y='total_bill', data=tips, palette='Set2')

# Violin plot – like boxplot but shows full distribution shape
sns.violinplot(x='day', y='total_bill', data=tips)

# Heatmap – great for correlation matrices
corr = tips.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')

# Pair plot – scatter matrix for all numeric columns
sns.pairplot(tips, hue='sex')

# Count plot – bar chart of category frequencies
sns.countplot(x='day', data=tips, palette='pastel')

# Regression plot – scatter + trendline
sns.regplot(x='total_bill', y='tip', data=tips)

plt.show()
```

Seaborn Plot	Best For
histplot(kde=True)	Distribution shape of one continuous variable
boxplot()	Comparing distributions; spotting outliers
violinplot()	Richer distribution shape than boxplot
heatmap()	Correlation matrix or 2-D grid data
pairplot()	All pairwise relationships in one figure
countplot()	Frequency of each category
regplot()	Scatter with automatic regression trendline
barplot()	Mean (with confidence interval) per category

■ `sns.set_theme()` at the start of your script sets a consistent aesthetic for all Seaborn plots. Use `hue=` parameter to colour-code by a categorical column.

12 · Plotly — Interactive Charts

Plotly generates **interactive HTML charts** — users can zoom, pan, hover for values, and toggle legend items. Ideal for dashboards and Jupyter notebooks.

```

import plotly.express as px

# Line chart
fig = px.line(df, x='Month', y='Revenue', title='Monthly Revenue')
fig.show()

# Bar chart
fig = px.bar(df, x='Category', y='Sales', color='Region',
             barmode='group', title='Sales by Category')
fig.show()

# Scatter plot
fig = px.scatter(df, x='Spend', y='Revenue', color='Region',
                size='Profit', hover_data=['Country'])
fig.show()

# Histogram
fig = px.histogram(df, x='Age', nbins=20, color='Gender')
fig.show()

# Heatmap
fig = px.imshow(corr_matrix, text_auto=True, color_continuous_scale='RdBu')
fig.show()

# Save as interactive HTML
fig.write_html('chart.html')

# Save as static image (needs kaleido)
fig.write_image('chart.png')

```

■ `plotly.express` (px) is the quick high-level API. `plotly.graph_objects` (go) gives full low-level control. Plotly charts work directly inside Jupyter notebooks — no `plt.show()` needed.

13 · Choosing the Right Chart

Goal	Chart Type	Library
Show trend over time	Line plot	plt / px
Compare categories	Bar chart	plt / sns / px
Show distribution of 1 variable	Histogram	plt / sns.histplot
Show distribution + outliers	Box / Violin plot	sns
Show relationship between 2 vars	Scatter plot	plt / sns / px
Show part-to-whole proportions	Pie / Donut chart	plt / px
Show correlation matrix	Heatmap	sns.heatmap
Explore all pairwise relationships	Pair plot	sns.pairplot
Show frequency of categories	Count plot	sns.countplot
Show geo data on a map	Choropleth / Scatter map	px.choropleth
Interactive dashboard	Any Plotly chart	px / go

Quick Reference — Visualisation Essentials

Function	Library	Purpose
<code>plt.plot(x, y)</code>	Matplotlib	Line chart
<code>plt.bar(x, height)</code>	Matplotlib	Vertical bar chart
<code>plt.barh(y, width)</code>	Matplotlib	Horizontal bar chart
<code>plt.hist(data, bins=n)</code>	Matplotlib	Histogram
<code>plt.scatter(x, y)</code>	Matplotlib	Scatter plot
<code>plt.pie(sizes, labels=...)</code>	Matplotlib	Pie chart
<code>plt.subplots(r, c)</code>	Matplotlib	Multi-panel figure
<code>plt.savefig('file.png')</code>	Matplotlib	Save chart to file
<code>sns.histplot(kde=True)</code>	Seaborn	Distribution + density curve
<code>sns.boxplot(x, y, data)</code>	Seaborn	Box plot
<code>sns.violinplot(...)</code>	Seaborn	Violin plot
<code>sns.heatmap(corr, annot=T)</code>	Seaborn	Correlation heatmap
<code>sns.pairplot(df, hue=...)</code>	Seaborn	All-pairs scatter matrix
<code>sns.regplot(x, y, data)</code>	Seaborn	Scatter + regression line
<code>px.line(df, x, y)</code>	Plotly	Interactive line chart
<code>px.bar(df, x, y)</code>	Plotly	Interactive bar chart
<code>px.scatter(df, x, y)</code>	Plotly	Interactive scatter plot
<code>px.histogram(df, x)</code>	Plotly	Interactive histogram
<code>fig.write_html('f.html')</code>	Plotly	Export as interactive HTML

```

# Full EDA visualisation pipeline
import pandas as pd, matplotlib.pyplot as plt, seaborn as sns

df = pd.read_csv('sales.csv')
sns.set_theme(style='whitegrid')

fig, axes = plt.subplots(2, 2, figsize=(12, 9))

# 1. Distribution of Revenue
sns.histplot(df['Revenue'], kde=True, ax=axes[0,0], color='steelblue')
axes[0,0].set_title('Revenue Distribution')

# 2. Revenue by Category
sns.boxplot(x='Category', y='Revenue', data=df, ax=axes[0,1])
axes[0,1].set_title('Revenue by Category')

# 3. Profit vs Revenue scatter
axes[1,0].scatter(df['Revenue'], df['Profit'], alpha=0.5, color='coral')
axes[1,0].set_title('Profit vs Revenue')

# 4. Monthly trend
df.groupby('Month')['Revenue'].sum().plot(ax=axes[1,1], marker='o')
axes[1,1].set_title('Monthly Revenue Trend')

plt.tight_layout()
plt.savefig('eda_dashboard.png', dpi=150, bbox_inches='tight')
plt.show()

```

■ This 2x2 dashboard — distribution, comparison, correlation, trend — is the standard EDA starting point for any new dataset. Always visualise before modelling.